server_event_block.c 3.39 KiB

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <sys/epoll.h>

#define PORT 9595
#define BUF_SIZE 1
#define FD_MAX 1024

const char *server_message = "Server Connect Success!\n";

void err_exit(const char *err){
    perror(err);
    exit(1);
}

void clear_fd(const int epoll_fd, const int fd){
    epoll_ctl(epoll_fd, EPOLL_CTL_DEL, fd, NULL);
    close(fd);
}

int main(void){
    int socket_fd, accepted_fd;
    struct sockaddr_in host_addr, client_addr;
    socklen_t size;
    int epoll_fd, i, msg_count, ret;
    char buffer[BUF_SIZE] = {0,};
    struct epoll_event events[FD_MAX];
    int pos = 0;

    socket_fd = socket(PF_INET, SOCK_STREAM, 0);
    if(socket_fd < 0) err_exit("socket error");
    memset(&host_addr, 0, sizeof(host_addr));
    host_addr.sin_family = AF_INET;
    host_addr.sin_port = htons(PORT);
    host_addr.sin_addr.s_addr = 0;

    if(bind(socket_fd, (struct sockaddr *)&host_addr, sizeof(struct sockaddr)) < 0)
        err_exit("bind error");
    if(listen(socket_fd, 3) < 0)
        err_exit("listen error");
    for(i = 0 ; i < FD_MAX ; i++)
        events[i].data.fd = -1;
    epoll_fd = epoll_create(FD_MAX);

    if(epoll_fd < 0) err_exit("epoll_create error");
    struct epoll_event socket_ev;
    socket_ev.data.fd = socket_fd;
    socket_ev.events = EPOLLIN;
    if(epoll_ctl(epoll_fd, EPOLL_CTL_ADD, socket_fd, &socket_ev) < 0)
        err_exit("epoll_ctl error");
    while(1){
        ret = epoll_wait(epoll_fd, events, FD_MAX, -1);
        if(ret == -1) err_exit("epoll_wait error");
        for(i = 0 ; i< ret; i++){
            if(events[i].data.fd == socket_fd && events[i].events & EPOLLIN){
                size = sizeof(struct sockaddr_in);
                accepted_fd = accept(socket_fd, (struct sockaddr*)&client_addr, &size);
                if(accepted_fd < 0)
                    err_exit("accept error");
                struct epoll_event client;
                client.data.fd = accepted_fd;
                client.events = EPOLLIN;
                if(epoll_ctl(epoll_fd, EPOLL_CTL_ADD, accepted_fd, &client) < 0)
                    err_exit("epoll_ctl error");
                printf("Client Info : IP %s, Port %d\n"
                    , inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));
                msg_count = send(accepted_fd, server_message, strlen(server_message), 0);
                if(msg_count < 0) err_exit("send error");
                continue;
            }

            if(events[i].events & EPOLLIN){
                msg_count = recv(events[i].data.fd,& buffer[pos], BUF_SIZE, 0);
                if(msg_count < 0) err_exit("recv_error");
                if(msg_count == 0 ){
                    clear_fd(epoll_fd, events[i].data.fd);
                    pos = 0;
                    continue;
                }
                if(buffer[pos] == '\0'){
                    if(!strcmp(buffer, "quit")){
                        clear_fd(epoll_fd, events[i].data.fd);
                        pos = 0;
                        continue;
                    }
                    msg_count = send(events[i].data.fd, buffer, pos, 0);
                    if(msg_count < 0) err_exit("send error");
                    pos = 0;
                }else{
                    pos++;
                }
            }
        }
    }

    shutdown(socket_fd, SHUT_RDWR);
    return 0;
}
```